VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi

A Internship Report on

"PROMPT ENGINEERING AND PYTHON PROGRAMMING"

Submitted in partial fulfilment of the requirements for the award of degree

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

By

Govind(3RB22CS030)

Under the guidance of

Prof.Priyanak mam

Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BHEEMANNA KHANDRE INSTITUTE OF TECHNOLOGY

BHALKI-585 328

2025-2026

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGNEERING

BHEEMANNA KHANDRE INSTITUTE OF TECHNOLOGY

BHALKI-585 328

CERTIFICATE

This is to certify that **Govind(3RB22CS043)** of 8th semester in Bachelors of Engineering in Computer Science and Engineering has successfully completed. The internship on "Prompt Engineering and Python Programming" and has submitted the report for partial fulfillment of the academic requirement of Visveswaraya Technological University, Belagavi during the year 2025-2026

Internal Guide	**HOD**	**PRINCIPAL**
Prof. Priyanka Dept of CSE	**Dr. Sangamesh Kalyane** Dept of CSE	**Dr. Udayakumar K** BKIT, Bhalki

Name of Examiner	Signature with Date
1. _____	_____
2. _____	_____

DECLARATION

1, the student of BE in **Computer Science and Engineering** Department **Bbeemanna Khandre Institute of Technology Bhalki-585328**, declare that the work on 12 weeks internship program entitled "**Prompt Engineering & Python Programming**" has been successfully completed under the guidance of **Dr. Sangamesh Kalyane** This work is submitted to **Visvesvaraya Technological University, Belagavi** in partial fulfillment of the requirements for award of B.E. m Computer Science and Engineering during the academic year 2025-2026. Further the matter embodied in the internship report has not been submitted previously by any body for the award of any degree or diploma to any university

place: **Bhalki** GOVIND(3RB22EC030)

Date:

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to the Management of Bharat Unnati AI Fellowship for organizing this internship program on **"Prompt Engineering and Python Programming"** and for providing a well-structured platform to enhance my technical knowledge and practical skills.

I am deeply indebted to **Dr. Udaykumar Kalyane**, Principal of Bheemanna Khandre Institute of Technology, for his visionary leadership, encouragement, and continuous support, which inspired me to successfully complete this internship.

I express my heartfelt gratitude to **Dr. Sangamesh Jayprakash Kalyane**, Head of the Department of Computer Science and Engineering, Bheemanna Khandre Institute of Technology, for his valuable guidance, motivation, and academic support throughout the internship period.

I would like to extend my sincere appreciation to the faculty members of the Department of Computer Science and Engineering, Bheemanna Khandre Institute of Technology, for their constant encouragement and for providing the strong academic foundation necessary to undertake this internship.

I am especially thankful to **Mr. Harsh Nath**, Director of Bharat Unnati, for his expert guidance, technical insights, and mentorship throughout the program. His support greatly contributed to my understanding of Prompt Engineering, Artificial Intelligence concepts, and Python Programming.

I also acknowledge the mentors and coordinators of Bharat Unnati AI Fellowship for designing an industry-oriented learning experience that enabled me to gain practical exposure and improve my problem-solving abilities.

I convey my heartfelt thanks to my parents and friends for their unwavering encouragement, moral support, and motivation during the successful completion of this internship.

Finally, I express my sincere gratitude to all individuals who, directly or indirectly, contributed to the successful completion of this internship and the preparation of this report.

Govind
USN: 3RB22CS030

CERTIFICATE



CONTENTS

Chapter No.	Description	Page No.
1	About the Organization	3
2	Introduction to Prompt Engineering	5
3	Python Programming Fundamentals	10
4	Large Language Models (LLMs)	16
5	AI Platforms and Tools	22
6	Practical Exercises and Mini Projects	27
7	Results and Discussion	33
	Conclusion	35
	References	36
	Attachments	37

LIST OF FIGURES

Figure No.	Description	Page No.
2.1	Prompt Engineering Workflow	5
2.2	Components of an Effective Prompt	7
3.1	Python Program Execution Flow	10
3.2	Python Data Types Overview	13
4.1	Architecture of a Large Language Model	16
4.2	Tokenization Process	18
5.1	Overview of AI Platforms Used	22
5.2	Comparison of ChatGPT, Gemini, and Claude	24
6.1	Prompt Testing and Iteration Process	27
6.2	Sample Project Workflow	30
7.1	Project Output Interface	33
8.1	Internship Completion Certificate	37

LIST OF TABLES

Table No.	Description	Page No.
2.1	Types of Prompting Techniques	8
3.1	Python Operators and Their Usage	14
3.2	Common Python Built-in Functions	15
4.1	Comparison of LLMs and Traditional NLP Models	19
5.1	AI Platforms and Their Features	25
5.2	Advantages and Limitations of AI Tools	26
6.1	Mini Projects Developed During Internship	31

CHAPTER 1

ABOUT THE ORGANIZATION

1.1 Introduction to Bharat Unnati AI Fellowship

Bharat Unnati AI Fellowship is an innovative educational initiative designed to empower students and young professionals with practical knowledge and industry-relevant skills in the field of Artificial Intelligence (AI). The fellowship aims to bridge the gap between academic learning and real-world applications by offering structured training programs, hands-on projects, expert mentorship, and career guidance.

The program is focused on building a strong foundation in emerging technologies such as Prompt Engineering, Python Programming, Machine Learning, Large Language Models (LLMs), Generative AI, and various AI-powered tools and platforms. Through this fellowship, participants gain exposure to cutting-edge technologies that are transforming industries worldwide.

Bharat Unnati AI Fellowship is committed to creating a learning ecosystem that encourages innovation, problem-solving, and continuous improvement. By combining theoretical concepts with practical implementation, the organization helps students develop the skills required to succeed in the rapidly evolving technology landscape.

1.2 Vision of the Organization

The vision of Bharat Unnati AI Fellowship is to create a technologically empowered generation capable of leveraging Artificial Intelligence to solve real-world problems and contribute to national and global development.

The organization strives to:

- Promote awareness and understanding of Artificial Intelligence and related technologies.
- Provide quality education accessible to students from diverse backgrounds.
- Encourage innovation and research in emerging technologies.
- Equip learners with practical skills aligned with industry requirements.
- Foster entrepreneurship and technological leadership.

1.3 Mission of the Organization

The mission of Bharat Unnati AI Fellowship is to provide structured and practical training in Artificial Intelligence and associated technologies through expert mentorship, project-based learning, and career-oriented programs.

The key objectives include:

- Delivering comprehensive technical training.
- Offering real-world project experience.
- Building strong programming and analytical skills.
- Enhancing problem-solving and critical-thinking abilities.
- Preparing students for internships, research, and employment opportunities.

1.4 Objectives of the Fellowship Program

The fellowship program is designed with the following objectives:

1. To introduce students to the fundamentals of Artificial Intelligence and Generative AI.
2. To develop proficiency in Prompt Engineering techniques.
3. To strengthen programming skills using Python.
4. To provide knowledge of Large Language Models and their applications.
5. To familiarize students with modern AI platforms such as OpenAI, Google, Anthropic, and Microsoft.
6. To encourage the development of practical applications and mini-projects.
7. To enhance communication and technical documentation skills.

1.5 Core Areas of Training

During the fellowship, participants receive training in several important domains of Artificial Intelligence and software development.

Prompt Engineering

Prompt Engineering involves designing clear and effective instructions to guide AI systems in generating accurate and relevant responses. Students learn techniques such as zero-shot prompting, few-shot prompting, chain-of-thought prompting, and role-based prompting.

Python Programming

Python serves as the primary programming language for AI and data-related applications. The fellowship covers variables, data types, control structures, functions, object-oriented programming, file handling, and libraries commonly used in AI development.

Large Language Models (LLMs)

Participants are introduced to the concepts behind LLMs, including tokenization, transformers, attention mechanisms, fine-tuning, and inference.

Generative AI Tools

Students gain hands-on experience with platforms such as OpenAI's ChatGPT, Google's Gemini, Anthropic's Claude, and other productivity-focused AI platforms.

1.6 Learning Methodology

Bharat Unnati AI Fellowship follows a practical and learner-centric methodology. The training process includes:

- Structured video lessons and guided tutorials.
- Live mentoring sessions.
- Assignments and coding exercises.
- Prompt experimentation and optimization.
- Mini-project development.
- Technical discussions and peer collaboration.
- Internship report preparation.

This methodology ensures that learners not only understand theoretical concepts but also gain confidence in applying them effectively.

1.7 Role of Mentors and Industry Experts

One of the most valuable aspects of the fellowship is the involvement of experienced mentors and industry professionals. They provide technical guidance, explain complex concepts in an accessible manner, and offer feedback on assignments and projects.

Under the leadership of Mr. Mallikarjun Kumbar, Director of Bharat Unnati, participants receive both technical support and career-oriented insights. This mentorship enables students to align their learning with current industry expectations.

1.8 Benefits of the Fellowship

Participation in Bharat Unnati AI Fellowship offers numerous benefits, including:

- Exposure to emerging AI technologies.
- Practical programming experience.
- Improved prompt design and experimentation skills.
- Enhanced understanding of LLMs and Generative AI.
- Development of mini-projects and technical portfolios.
- Industry mentorship and networking opportunities.
- Increased confidence for placements and future research.

1.9 Organizational Impact

Bharat Unnati AI Fellowship plays a significant role in nurturing technological talent among students. By offering accessible and industry-focused education, the organization contributes to workforce development and promotes innovation.

The fellowship equips students with practical skills that are increasingly important across sectors such as healthcare, finance, education, software development, and research.

1.10 Conclusion

Bharat Unnati AI Fellowship is a forward-looking organization dedicated to empowering students with modern Artificial Intelligence skills. Through structured learning, practical implementation, and expert mentorship, the organization provides a valuable platform for academic and professional growth.

The internship offered by Bharat Unnati AI Fellowship enabled me to gain in-depth knowledge of Prompt Engineering, Python Programming, Large Language Models, and AI tools. This experience significantly strengthened my technical capabilities and prepared me for future opportunities in the field of Artificial Intelligence and software development

CHAPTER 2

INTRODUCTION TO PROMPT ENGINEERING

2.1 Introduction

Prompt Engineering is the process of designing and refining instructions given to Artificial Intelligence (AI) systems in order to obtain accurate, relevant, and context-aware responses. A prompt is the input provided to a language model, and the quality of that prompt directly influences the quality of the generated output.

With the rapid advancement of Generative AI and Large Language Models (LLMs), Prompt Engineering has emerged as an essential skill for developers, researchers, content creators, and business professionals. By carefully structuring prompts, users can guide AI systems to perform tasks such as answering questions, generating code, summarizing documents, translating languages, and solving complex problems.

Prompt Engineering combines technical understanding, logical thinking, and experimentation. It involves selecting the right wording, context, examples, and constraints to obtain the desired results. This discipline has become increasingly important as AI tools are adopted across software development, education, healthcare, finance, and other industries.

2.2 What is a Prompt?

A prompt is a set of instructions, questions, or contextual information provided to an AI model to generate a response. Prompts can range from a simple question to a highly detailed instruction containing examples, formatting rules, and constraints.

Examples of Prompts

- “Explain Python functions in simple terms.”
- “Write a Java program to reverse a string.”
- “Summarize the following article in five bullet points.”
- “Act as a technical interviewer and ask five coding questions.”

A well-constructed prompt clearly communicates:

- The task to be performed.
- Relevant context.
- Desired output format.
- Constraints or conditions.

2.3 Importance of Prompt Engineering

Prompt Engineering is important because it significantly improves the reliability and usefulness of AI-generated outputs. Even highly advanced language models can produce incomplete or inaccurate responses when given vague instructions.

Benefits of Prompt Engineering

- Produces more accurate and relevant responses.
- Reduces ambiguity and misunderstandings.
- Improves consistency across repeated tasks.
- Saves time by minimizing prompt revisions.
- Enables automation of complex workflows.
- Enhances code generation and debugging.
- Supports professional documentation and content creation.

2.4 Components of an Effective Prompt

An effective prompt generally contains several important components:

Task Definition

Specifies exactly what the model should do.

Context

Provides background information to help the model understand the problem.

Constraints

Defines limitations such as word count, tone, or formatting rules.

Examples

Shows sample inputs and outputs.

Output Format

Specifies whether the response should be a paragraph, list, table, JSON object, or source code.

Role Assignment

Directs the model to respond from a specific perspective.

2.5 Prompt Engineering Workflow

The typical workflow includes:

1. Define the objective.
2. Write an initial prompt.
3. Test the prompt.
4. Analyze the response.
5. Refine the wording.
6. Add examples or constraints.
7. Repeat until the desired output is achieved.

This iterative approach is essential for obtaining high-quality responses.

2.6 Types of Prompting Techniques

Zero-Shot Prompting

The model performs the task without examples.

Few-Shot Prompting

A few examples are included to demonstrate the desired pattern.

Chain-of-Thought Prompting

The model is instructed to reason step by step.

Role-Based Prompting

The model is assigned a role such as teacher, engineer, or reviewer.

Structured Prompting

The output is constrained to a specific format such as JSON or Markdown.

Self-Consistency Prompting

Multiple reasoning paths are generated and compared to improve reliability.

2.7 Prompt Engineering Best Practices

- Use clear and precise instructions.
- Provide sufficient context.
- Specify the desired output format.
- Break complex tasks into smaller steps.
- Include examples when needed.
- State assumptions and constraints explicitly.
- Iterate and refine prompts based on results.
- Verify important outputs manually.

2.8 Applications of Prompt Engineering

Prompt Engineering is widely used in:

- Content generation
- Software development
- Educational tutoring
- Data extraction
- Summarization
- Translation

- Customer support
- Research assistance
- Business automation

2.9 Common Challenges

- Ambiguous instructions
- Incomplete context
- Hallucinated information
- Inconsistent formatting
- Overly broad prompts

These issues can usually be reduced by adding context, constraints, and examples.

2.10 Prompt Engineering in Software Development

Developers use Prompt Engineering to:

- Generate boilerplate code.
- Debug errors.
- Write unit tests.
- Refactor code.
- Generate API documentation.
- Design database schemas.

This increases productivity and accelerates development.

2.11 Real-World Example

Basic Prompt

“Write a Python function to sort a list.”

Improved Prompt

“Write a Python function named `sort_numbers` that accepts a list of integers and returns the list sorted in ascending order. Include type hints, comments, and an example usage.”

The improved prompt produces more structured and useful output.

2.12 Tools Used for Prompt Engineering

Common AI platforms used during the internship included:

- ChatGPT by OpenAI
- Gemini by Google
- Claude by Anthropic
- GitHub Copilot by GitHub

2.13 Conclusion

Prompt Engineering is a foundational skill in the era of Generative AI. It enables users to communicate effectively with Large Language Models and obtain accurate, structured, and meaningful outputs. Through this internship, I developed a practical understanding of prompt design, prompting strategies, and optimization techniques, which will be highly valuable in future academic and professional work involving Artificial Intelligence.

CHAPTER 3

PYTHON PROGRAMMING FUNDAMENTALS

3.1 Introduction

Python is a high-level, interpreted, and general-purpose programming language known for its simple syntax and readability. It was created by Guido van Rossum and first released in 1991. Python has become one of the most widely used programming languages in the world due to its versatility and extensive ecosystem of libraries.

Python is used in a broad range of applications, including web development, data analysis, machine learning, automation, cybersecurity, scientific computing, and Artificial Intelligence. During this internship, Python served as the primary programming language for implementing examples, experimenting with AI concepts, and developing mini-projects.

3.2 Features of Python

Python offers several features that make it an ideal language for beginners and professionals alike.

Simple and Readable Syntax

Python uses clear and concise syntax, making programs easy to write and understand.

Interpreted Language

Python code is executed line by line, which simplifies debugging and testing.

Dynamically Typed

Variables do not require explicit type declarations.

Cross-Platform Support

Python programs can run on Windows, Linux, and macOS without significant changes.

Extensive Libraries

Python includes a rich standard library and thousands of third-party packages.

Open Source

Python is freely available and maintained by a large global community.

Integration Capabilities

Python can be integrated with other languages and systems.

3.3 Installation and Development Tools

Python can be downloaded from Python.org. During the internship, several development tools were used:

- Visual Studio Code
- Jupyter Notebook
- PyCharm
- Command-line terminal

These tools provided an efficient environment for writing, executing, and debugging Python code.

3.4 Variables and Data Types

Variables are used to store data in memory.

Common Data Types

- `int` – Integer values
- `float` – Decimal numbers
- `str` – Text strings
- `bool` – Boolean values (`True` or `False`)
- `list` – Ordered collection of items
- `tuple` – Immutable ordered collection
- `set` – Unordered collection of unique elements
- `dict` – Key-value pairs

Example

name = "Mohan"

age = 21

marks = 92.5

is_student = True

3.5 Operators in Python

Python supports several categories of operators:

- Arithmetic Operators (`+`, `-`, `*`, `/`, `%`, `//`, `**`)
- Comparison Operators (`==`, `!=`, `>`, `<`, `>=`, `<=`)
- Logical Operators (`and`, `or`, `not`)
- Assignment Operators (`=`, `+=`, `-=`)
- Membership Operators (`in`, `not in`)
- Identity Operators (`is`, `is not`)

3.6 Control Statements

Control statements determine the flow of execution.

Conditional Statements

- `if`
- `elif`
- `else`

Looping Statements

- `for`
- `while`

Loop Control

- `break`
- `continue`
- `pass`

3.7 Functions

Functions are reusable blocks of code that perform specific tasks.

Example

```
def greet(name):
    return f"Hello, {name}"
```

Functions improve modularity, readability, and code reuse.

3.8 Data Structures

Python provides built-in data structures for organizing information.

Lists

Mutable ordered collections.

Tuples

Immutable ordered collections.

Sets

Unordered collections with unique values.

Dictionaries

Mappings between keys and values.

These structures are fundamental for storing and processing data.

3.9 String Handling

Strings are sequences of characters and support numerous operations:

- Concatenation
- Slicing
- Searching
- Formatting
- Case conversion

Example

```
message = "Prompt Engineering"

print(message.upper())
```

3.10 File Handling

Python can read from and write to files.

Common Modes

- `r` – Read
- `w` – Write

- a – Append

Example

```
with open("notes.txt", "w") as file:
    file.write("Python is powerful.")
```

3.11 Exception Handling

Exceptions are runtime errors that can be handled gracefully.

Example

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid input")
```

This prevents abrupt program termination.

3.12 Object-Oriented Programming (OOP)

Python supports OOP concepts such as:

- Classes and Objects
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

OOP is useful for designing scalable and maintainable applications.

3.13 Modules and Packages

Modules are Python files containing reusable code. Packages are collections of modules.

Common Modules Used

- `math`
- `random`
- `os`
- `json`
- `datetime`

Third-party packages can be installed using `pip`.

3.14 Python Libraries for AI and Data Science

Python has powerful libraries for AI and data processing:

- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- TensorFlow
- PyTorch

3.15 Role of Python in Prompt Engineering and AI

Python is widely used to:

- Interact with AI APIs
- Process and clean text
- Automate prompt testing
- Parse structured outputs
- Build web applications and chatbots

During the internship, Python was used to implement practical examples and automate AI-related tasks.

3.16 Mini Programs Developed

Examples of programs practiced during the internship include:

- Calculator application
- Number guessing game
- File reader and writer
- Data structure exercises
- API integration examples

These programs strengthened core programming concepts.

3.17 Advantages of Python

- Easy to learn
- Highly readable
- Large community support
- Extensive libraries
- Suitable for AI and automation
- Rapid development

3.18 Conclusion

Python is a powerful and versatile programming language that forms the foundation for modern Artificial Intelligence development. Through this internship, I developed a solid understanding of Python programming fundamentals, including variables, functions, data structures, file handling, and object-oriented programming. These skills are essential for building AI applications and solving real-world problems efficiently.

CHAPTER 4

LARGE LANGUAGE MODELS (LLMs)

4.1 Introduction

Large Language Models (LLMs) are advanced Artificial Intelligence systems trained on vast amounts of text data to understand and generate human-like language. These models are capable of performing a wide range of tasks, including answering questions, summarizing documents, generating source code, translating languages, and assisting with research and decision-making.

LLMs represent a major breakthrough in Natural Language Processing (NLP) and form the foundation of modern Generative AI applications. By learning patterns, grammar, reasoning structures, and contextual relationships from massive datasets, LLMs can generate coherent and contextually relevant responses.

During this internship, I studied the working principles of LLMs, their architecture, tokenization, attention mechanisms, training processes, and practical applications in software development and Prompt Engineering.

4.2 Evolution of Language Models

Language models have evolved significantly over the years.

Rule-Based Systems

Early systems relied on predefined rules and manually written logic.

Statistical Models

Models such as n-grams estimated word probabilities using frequency counts.

Neural Networks

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks improved contextual understanding.

Transformer Models

The Transformer architecture revolutionized NLP by introducing self-attention.

Large Language Models

Modern LLMs are trained on large-scale datasets and support general-purpose language understanding and generation.

4.3 What Makes an LLM “Large”

The term “large” refers to:

- Billions or trillions of model parameters.
- Training on massive datasets.
- Significant computational resources.
- Broad generalization capabilities.

The large scale enables sophisticated reasoning, language understanding, and knowledge representation.

4.4 Tokens and Tokenization

LLMs process text as tokens rather than raw characters or words.

Token

A token may represent a word, subword, punctuation mark, or symbol.

Tokenization

The process of converting text into tokens.

For example, the sentence “Python is powerful” may be split into several tokens before being processed by the model.

Tokenization affects context length, cost, and performance.

4.5 Transformer Architecture

The Transformer architecture, introduced by Ashish Vaswani and colleagues, is the foundation of modern LLMs.

Key Components

- Embedding Layer
- Positional Encoding
- Multi-Head Self-Attention
- Feed-Forward Neural Networks
- Layer Normalization

The transformer enables efficient parallel processing and strong contextual understanding.

4.6 Attention Mechanism

The attention mechanism allows the model to focus on the most relevant tokens when generating each output token.

Benefits include:

- Better handling of long-range dependencies.
- Improved contextual understanding.
- Enhanced translation and reasoning performance.

Attention is one of the most important innovations in modern AI.

4.7 Training Process

LLMs are typically trained in multiple stages.

Pretraining

The model learns general language patterns by predicting missing or next tokens.

Fine-Tuning

The model is adapted to specific tasks or domains.

Alignment

Human feedback and reinforcement techniques improve helpfulness and safety.

4.8 Context Window

The context window defines how much text the model can consider at one time.

A larger context window enables:

- Analysis of long documents.
- Multi-step conversations.
- Retrieval-augmented workflows.
- Better memory within a session.

4.9 Inference

Inference is the process of generating outputs after training is complete.

Important generation parameters include:

- Temperature
- Top-p
- Maximum tokens
- Stop sequences

These parameters control creativity, randomness, and response length.

4.10 Capabilities of LLMs

LLMs can perform numerous tasks, including:

- Question answering
- Summarization
- Translation
- Code generation
- Document analysis
- Chat-based assistance
- Data extraction
- Educational tutoring

4.11 Limitations of LLMs

Despite their capabilities, LLMs have limitations.

- Hallucinations (incorrect facts)
- Bias inherited from training data
- Limited real-time knowledge unless connected to tools
- Sensitivity to prompt wording
- Computational cost

Human verification remains essential for critical applications.

4.12 Popular LLMs

Several leading organizations have developed powerful LLMs.

- ChatGPT by OpenAI
- Gemini by Google
- Claude by Anthropic
- Llama by Meta Platforms
- Mistral by Mistral AI

4.13 Applications in Industry

LLMs are used across many sectors:

- Software development
- Customer support
- Healthcare documentation
- Legal analysis
- Education
- Finance
- Research
- Content creation

Their versatility makes them valuable across industries.

4.14 Role of LLMs in Prompt Engineering

Prompt Engineering is the primary method of communicating with LLMs. Well-structured prompts improve response quality, accuracy, and consistency.

Through prompt design, users can guide models to:

- Follow specific instructions
- Return structured data
- Perform step-by-step reasoning
- Adopt specialized roles

4.15 LLMs and APIs

Many LLMs are accessible through APIs, allowing developers to integrate them into applications.

Typical use cases include:

- Chatbots
- Code assistants
- Document analyzers
- Automation systems

Python is commonly used to interact with these APIs.

4.16 Ethical Considerations

Responsible use of LLMs requires attention to:

- Privacy
- Security
- Bias mitigation
- Transparency
- Human oversight

Ethical deployment is essential for trustworthy AI systems.

4.17 Conclusion

Large Language Models are a transformative technology that enables computers to understand and generate human language with remarkable effectiveness. During this internship, I gained a strong understanding of how LLMs work, including tokenization, transformers, attention mechanisms, training, and inference. This knowledge provided a solid foundation for applying Prompt Engineering and Generative AI techniques in real-world applications.

CHAPTER 5

AI PLATFORMS AND TOOLS USED DURING THE INTERNSHIP

5.1 Introduction

During the internship, I gained practical exposure to a wide range of Artificial Intelligence platforms, software development tools, and productivity applications. These tools played a vital role in helping me understand Prompt Engineering, Python Programming, Large Language Models (LLMs), workflow automation, and Generative AI applications. By working with these platforms, I was able to explore how AI systems can be used to improve productivity, automate repetitive tasks, and solve complex technical problems.

The internship emphasized hands-on learning, and each tool contributed to a specific aspect of the training. Some platforms were used for prompt experimentation and content generation, while others were used for coding, automation, technical documentation, and presentation preparation. This practical experience significantly enhanced my technical knowledge and problem-solving skills.

5.2 Artificial Intelligence Platforms

ChatGPT

ChatGPT, developed by OpenAI, was the primary AI platform used throughout the internship. It was utilized for prompt design, concept explanation, code generation, debugging, report writing, and project planning. The platform demonstrated the importance of well-structured prompts and provided instant feedback for experimentation.

Gemini

Gemini, developed by Google, was used to compare the performance of different AI models. It provided assistance with research, content generation, and multimodal analysis involving text and images.

Claude

Claude, developed by Anthropic, was used for summarization, document analysis, and generating detailed long-form responses. It was especially useful for working with lengthy documents and structured content.

GitHub Copilot

GitHub Copilot, developed by GitHub, assisted in writing Python code, suggesting functions, generating comments, and improving coding efficiency.

5.3 Workflow Automation and Productivity Tools

n8n

n8n is an open-source workflow automation platform that enables users to connect APIs, databases, and third-party services without extensive manual coding. During the internship, I explored how n8n can integrate AI models with applications to automate tasks such as sending notifications, processing data, and triggering workflows based on specific conditions.

Microsoft PowerPoint

Microsoft PowerPoint was used to create technical presentations summarizing internship concepts, prompt engineering techniques, and project demonstrations. It helped present information in a structured and visually effective manner.

Google Docs

Google Docs was used to prepare and format the internship report, including the title page, tables, figures, and chapter content.

5.4 Programming and Development Tools

Visual Studio Code

Visual Studio Code was the primary editor used for writing Python programs and experimenting with code examples. Its extensions and debugging tools made development more efficient.

Jupyter Notebook

Jupyter Notebook was used for executing Python code interactively and documenting results step by step.

GitHub

GitHub was used for version control, source code management, and maintaining a portfolio of projects completed during the internship.

Python.org

The official Python website was used to download Python and access documentation and tutorials.

5.5 Applications of the Tools

The platforms and tools were applied in several practical areas:

- Designing and optimizing prompts.
- Generating and debugging Python programs.
- Understanding LLM behavior.
- Automating workflows using n8n.
- Creating technical presentations.
- Managing source code and project versions.
- Preparing professional documentation.
- Conducting research and comparative analysis.

This combination of tools provided an end-to-end understanding of how modern AI systems are used in real-world software development and automation.

5.6 Benefits of Learning These Platforms

Working with these tools provided numerous benefits:

- Improved productivity and efficiency.
- Enhanced coding and debugging skills.
- Better understanding of AI model behavior.
- Practical exposure to automation technologies.
- Stronger documentation and presentation skills.
- Increased confidence in building AI-powered applications.
- Familiarity with industry-standard software.

These benefits will support future academic projects, internships, and professional development.

5.7 Conclusion

The internship provided comprehensive hands-on experience with a diverse set of Artificial Intelligence platforms and development tools, including ChatGPT, Gemini, Claude, GitHub Copilot, n8n, Microsoft PowerPoint, Python, Visual Studio Code, Jupyter Notebook, GitHub, and Google Docs. Each tool contributed to a specific aspect of learning, ranging from prompt design and programming to automation and documentation.

This exposure significantly enhanced my technical capabilities and demonstrated how AI can be integrated into coding, research, communication, and business workflows. The knowledge gained from these platforms will be highly valuable in my future work in Artificial Intelligence and software development.

CHAPTER 6

PRACTICAL EXERCISES AND MINI PROJECTS

6.1 Introduction

A major component of the internship was the completion of practical exercises and mini projects designed to reinforce the concepts learned throughout the training. While theoretical knowledge provided the foundation, hands-on implementation enabled me to understand how Prompt Engineering, Python Programming, Large Language Models (LLMs), and workflow automation are applied in real-world scenarios.

The practical sessions were structured to help me develop technical proficiency through experimentation, coding, testing, debugging, and documentation. Each exercise was carefully designed to strengthen problem-solving skills and to demonstrate how Artificial Intelligence can be integrated into software development and automation.

Through these activities, I gained valuable experience in creating prompts, writing Python programs, building automation workflows using n8n, and utilizing AI platforms to enhance productivity.

6.2 Prompt Engineering Exercises

Prompt Engineering exercises formed the foundation of the practical training. These activities focused on designing, testing, and refining prompts to obtain accurate and structured responses from AI systems.

Zero-Shot Prompting

I created prompts that instructed the model to perform tasks without examples. This helped me understand how clearly defined instructions influence output quality.

Few-Shot Prompting

By providing a small number of examples, I observed how models learn desired patterns and generate more consistent results.

Role-Based Prompting

I assigned roles such as “Python Instructor,” “Software Architect,” and “Technical Writer” to guide the tone and depth of responses.

Chain-of-Thought Prompting

I used prompts that requested step-by-step reasoning, which improved the quality of logical explanations and problem-solving.

Structured Output Prompting

I generated responses in specific formats such as bullet points, Markdown tables, JSON objects, and code blocks.

Prompt Refinement

I iteratively modified prompts by adding context, constraints, and examples to improve response relevance and accuracy.

These exercises demonstrated that Prompt Engineering is an iterative process requiring clarity, experimentation, and critical evaluation.

6.3 Python Programming Exercises

Python programming exercises were conducted to build a strong foundation in software development and automation.

Basic Programs

I developed simple programs such as:

- Arithmetic calculator
- Temperature converter
- Number guessing game
- Prime number checker
- Fibonacci series generator

String Processing

Programs were written to:

- Reverse strings
- Count vowels and consonants
- Check palindromes
- Format text output

Data Structures

I practiced using:

- Lists
- Tuples
- Sets
- Dictionaries

Functions and Modules

I created reusable functions and organized code into modules.

File Handling

Programs were developed to read from and write to text files.

Exception Handling

I implemented `try`, `except`, and `finally` blocks to manage runtime errors gracefully.

Object-Oriented Programming

I created classes and objects to model real-world entities and apply principles such as inheritance and encapsulation.

These exercises improved my coding style, debugging ability, and understanding of software design principles.

6.4 Large Language Model Experiments

To understand the behavior of LLMs, I performed several experiments using AI platforms such as ChatGPT, Gemini, and Claude.

Comparative Prompt Testing

The same prompt was executed across multiple platforms to compare accuracy, structure, and reasoning quality.

Token and Context Experiments

I explored how prompt length and context affected response relevance.

Code Generation Tests

I used natural language prompts to generate Python programs and evaluated the correctness of the output.

Summarization and Extraction

I tested document summarization, key-point extraction, and conversion of text into structured data.

Report Writing Assistance

AI tools were used to generate and refine sections of technical documentation.

These experiments provided a practical understanding of model strengths, limitations, and prompt sensitivity.

6.5 Workflow Automation Using n8n

As part of the internship, I explored workflow automation using n8n.

Workflow Concepts Studied

- Triggers
- Nodes
- Conditional logic
- API calls
- Data transformations

Sample Workflows Created

- Receiving input from a form.
- Sending data to an AI model.
- Processing the generated response.
- Saving the result.
- Sending notifications.

Benefits Observed

- Reduced manual effort.
- Faster execution of repetitive tasks.
- Integration of multiple services.
- Scalable automation solutions.

This practical exposure showed how AI can be combined with automation platforms to create efficient business workflows.

6.6 Mini Projects Developed

Several mini projects were completed to integrate the concepts learned during the internship.

1. AI-Powered Content Generator

Designed prompts to generate summaries, explanations, and formatted technical content.

2. Python Utility Collection

Developed a set of scripts for file handling, text processing, and automation.

3. Prompt-Based Coding Assistant

Used AI to generate, review, and optimize Python code based on natural language instructions.

4. Automated Workflow Prototype

Created an n8n workflow to connect user input, AI processing, and automated notifications.

5. Internship Report Preparation

Used AI tools and Google Docs to organize and format a professional technical report.

Each project contributed to my understanding of how multiple technologies can be combined to solve practical problems.

6.7 Challenges Encountered

During the practical sessions, several challenges were encountered:

- Designing prompts that produced consistent outputs.
- Debugging Python syntax and logic errors.
- Handling incomplete or incorrect AI-generated responses.
- Understanding automation workflow configurations.
- Integrating multiple tools effectively.

These challenges were addressed through repeated experimentation, testing, and mentor guidance.

6.8 Skills Developed

The exercises and projects helped me develop a broad range of technical and professional skills:

Technical Skills

- Prompt Engineering
- Python Programming
- AI tool usage
- Workflow automation
- Debugging
- Documentation

Analytical Skills

- Logical thinking
- Comparative evaluation
- Error analysis
- Problem decomposition

Professional Skills

- Report writing
- Presentation preparation
- Time management
- Independent learning

6.9 Overall Learning Experience

The practical component of the internship transformed theoretical concepts into applicable skills. I gained confidence in interacting with AI models, writing Python programs, automating workflows, and developing small-scale solutions. The structured exercises and projects provided a comprehensive understanding of how modern AI tools are used in real-world environments.

6.10 Conclusion

The practical exercises and mini projects were among the most valuable aspects of the internship. They enabled me to apply Prompt Engineering, Python Programming, Large Language Models, and workflow automation to real-world scenarios. Through continuous experimentation and project development, I strengthened my technical skills, enhanced my creativity, and developed a deeper understanding of Artificial Intelligence and software engineering practices.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Introduction

The internship on **Prompt Engineering and Python Programming** provided a comprehensive learning experience that combined theoretical knowledge with practical implementation. Through structured training, hands-on exercises, and mini projects, I developed a strong understanding of Artificial Intelligence, Large Language Models (LLMs), Python programming, and workflow automation.

This chapter presents the results obtained during the internship and discusses the knowledge, technical skills, and practical competencies gained throughout the program.

7.2 Results Obtained

The internship produced several significant outcomes that enhanced my technical proficiency and problem-solving abilities.

Knowledge Gained

During the internship, I acquired a thorough understanding of:

- Prompt Engineering principles and prompting strategies.
- Python programming fundamentals and advanced concepts.
- Large Language Models and transformer-based architectures.
- AI platforms such as ChatGPT, Gemini, and Claude.
- Workflow automation using n8n.
- Technical documentation and presentation preparation.

Practical Skills Developed

The following practical skills were strengthened:

- Designing and refining effective prompts.
- Writing, debugging, and optimizing Python programs.
- Generating structured outputs from AI systems.
- Building automated workflows.
- Creating technical reports and presentations.
- Managing code using Git and GitHub.

Mini Projects Completed

The successful completion of multiple exercises and mini projects demonstrated my ability to apply the concepts learned during the internship to real-world tasks.

7.3 Discussion

The internship highlighted the growing importance of Prompt Engineering and Artificial Intelligence in modern software development. One of the key observations was that the quality of AI-generated output depends heavily on the clarity and structure of the prompt. By iteratively refining prompts, it was possible to significantly improve response accuracy and consistency.

Python programming proved to be an essential skill for implementing automation scripts, processing data, and integrating AI models into practical applications. The use of AI platforms accelerated coding, documentation, and research activities, demonstrating how these technologies can enhance productivity.

The exploration of n8n illustrated how AI can be combined with workflow automation to create scalable and efficient solutions. This integration opens new opportunities for developing intelligent systems that reduce manual effort and streamline business processes.

The internship also emphasized the importance of validating AI-generated outputs, as responses may occasionally contain inaccuracies or incomplete information. Human review and critical thinking remain essential components of responsible AI usage.

7.4 Overall Outcome

The overall outcome of the internship was highly positive. I successfully completed all assigned modules, practical exercises, and project activities. The training improved my technical knowledge, increased my confidence in using AI tools, and provided a strong foundation for future projects in Artificial Intelligence and software development.

The internship also enhanced my communication, documentation, and presentation skills, which are essential for both academic and professional success.

7.5 Conclusion

The results of this internship demonstrate that practical exposure to Prompt Engineering, Python Programming, Large Language Models, and automation tools significantly enhances technical competence and problem-solving ability. The knowledge and skills gained during this program will be highly valuable in my future academic work, internships, and career in Artificial Intelligence and software development.

CONCLUSION

The internship on **“Prompt Engineering and Python Programming”** conducted through the Bharat Unnati AI Fellowship was an enriching and highly valuable learning experience. It provided me with comprehensive exposure to emerging technologies in Artificial Intelligence, Large Language Models (LLMs), workflow automation, and software development.

Throughout the internship, I gained a strong understanding of Prompt Engineering techniques, including zero-shot prompting, few-shot prompting, role-based prompting, chain-of-thought reasoning, and structured output generation. These concepts helped me understand how carefully designed prompts can significantly improve the quality and accuracy of AI-generated responses.

The internship also strengthened my Python programming skills. I developed practical programs involving variables, control structures, functions, file handling, exception handling, and object-oriented programming. This knowledge enabled me to automate tasks, interact with APIs, and build small-scale applications efficiently.

In addition, I studied the fundamentals of Large Language Models, including tokenization, transformer architecture, attention mechanisms, training, and inference. I gained hands-on experience with leading AI platforms such as ChatGPT, Gemini, Claude, GitHub Copilot, and n8n, which enhanced my ability to apply AI in practical scenarios.

The practical exercises and mini projects played a crucial role in converting theoretical concepts into real-world skills. They improved my problem-solving ability, analytical thinking, coding proficiency, and confidence in building AI-assisted applications and automation workflows.

Overall, this internship has significantly broadened my technical knowledge and prepared me to pursue advanced projects and career opportunities in Artificial Intelligence, Generative AI, and software development. The experience has inspired me to continue learning and exploring innovative technologies that can contribute to solving real-world problems effectively and efficiently.